2024 6th International Conference on Communications, Information System and Computer Engineering (CISCE) | 979-8-3503-5317-4/24/$31.00 ©2024 IEEE | DOI: 10.1109/CISCE62493.2024.10653209

Optimization of Macro Placement Using Genetic Algorithm

Xiaoxue Zhang
Engineering Research Center of Optoelectronic Devices and Communication Technology Tianjin University of Technology, Ministry of Education
School of Integrated Circuit Science and Engineering, Tianjin University of Technology
Tianjin, China

Huimin Liu*
Engineering Research Center of Optoelectronic Devices and Communication Technology Tianjin University of Technology, Ministry of Education
School of Integrated Circuit Science and Engineering, Tianjin University of Technology
Tianjin, China
*lhmdxx@tjut.edu.cn

Qiang Dou*
Phytium Technology Co., Ltd
Tianjin, China
*mark@phytium.com.cn

Zhuo Ma
Phytium Technology Co., Ltd
Tianjin, China

Longwen Wei
Phytium Technology Co., Ltd
Tianjin, China

***Abstract*—This thesis presents a novel approach to macro-layout in physical design, combining genetic algorithms with traditional design techniques. Genetic algorithms simulate the process of genetic evolution by natural selection, whereby the most environmentally adapted individuals are obtained through the evolution of reproduction from generation to generation. In this paper, the macro layout is represented by individuals, and the characteristics of the macro layout are used to constrain it. In the algorithm presented in this paper, each individual's adaptation action to their environmental context is used to represent the line length. The smaller the line length, the greater the macro layout's conformity to its constraints. Finally, the physical design of different macro layouts is compared with the timing obtained by STA tool. The final conversion time violation of macro layout applying the algorithm of this paper achieved an average reduction of 6.8% in violation and 17% in the number of violation paths compared to manual macro layout. The macro layout has been optimized.**

Keywords-physical design; macro placement; genetic algorithm

I. Introduction

In modern industry, layout methods are typically divided into two stages: macro placement and standard cell placement. In the former, macros are placed in legal locations around the chip boundaries, while in the latter, standard cells are placed in the remaining space after all macros have been fixed[1]. As illustrated in Fig. 1, this process is repeated until the desired layout is achieved. However, the evaluation of layout results is typically conducted manually only after the wiring step, which can lead to multiple iterations in the physical design flow and a reduction in overall productivity. As machine learning AI algorithms continue to gain popularity and are increasingly applied in various fields, there has been a proliferation of machine learning-based algorithms that utilize machine learning to accelerate the physical design process.

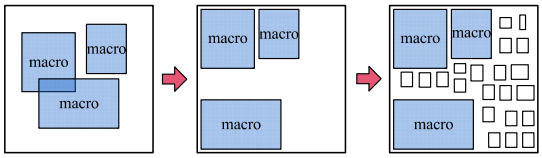

Figure 1. Basic macro placement flow

The macro layout ensures that all macros are situated within the boundaries of the chip and that no two macros overlap[2]. During the process of placing macros, it is crucial to prioritize the pin positions of macros, as the direction of pin placement affects the alignment[3].

Currently, standard layout algorithms can be classified into three categories[4]: (1) Placing macros and standard cells together. An example of this is the Mixed Placer. (2) Constructively placed macros. This placer can prevent macros from overlapping by recursively dividing the circuit into subregions. The min-cut floor placer, or the CP-tree[5], is a more robust placer type typically limited in solution quality, particularly in line length. (3) Mixed-size layouts, also known as macro and standard cell layouts, involve first determining the locations of the macros and then placing the standard cells into the remaining regions. An example of this type of placer is ePlace-MS[6].

This paper contributes the following: The algorithm presented in this paper can address the macro overlapping and macro misalignment issues that arise during macro placement. The algorithm employs a simulation approach, processing the macro information within the module to identify the optimal

Authorized licensed use limited to: The University of British Columbia Library. Downloaded on April 01,2026 at 02:49:29 UTC from IEEE Xplore. Restrictions apply.

macro layout for shorter line lengths. By optimizing the alignment length through the use of a genetic algorithm, the required materials (e.g., metal layers) can be reduced without compromising performance.

In this paper, we first introduce the difficulties of macro layout and some macro layouts. Section II describes the basic genetic algorithm and the genetic algorithm flow based on the macro layout in this paper, Section III describes the experimental procedure, Section IV shows the experimental results, and Section V gives the summary.

II. Algorithm Flow Analysis

A. Basic Genetic Algorithm

The Genetic Algorithm (GA) was developed from the computer simulation of biological systems. It is a stochastic global search optimization method that simulates the phenomena of replication, crossover, and mutation in natural selection and inheritance. It begins with an initial population. A group of individuals better suited to the environment is generated by applying random selection, crossover, and mutation operations. This group evolves over successive generations, gradually moving towards an increasingly optimal region in the search space. Ultimately, the group converges on one individual, the most adapted to the environment. This leads to a solution of optimal quality to the problem.

B. Genetic Algorithm and Macro Placement

During the physical design process, the position of the macro is placed manually, so there are many kinds of placement results. Different macro layouts can have a big impact on the line length and timing of the chip. There are many algorithms for macro layout, but few can think of the search and optimization capabilities of genetic algorithms. The genetic algorithm has excellent stochastic global search optimization ability, and the algorithm can find the better quality macro layout in advance, which can save labor and time, and can help engineers optimize the macro layout scheme.

C. Based on the Macro Placement Problem

This paper employs the genetic algorithm to identify potential macro layouts. The length of the lines typically assesses the quality of a macro layout. A shorter line length indicates a higher-quality macro layout. The genetic algorithm employs a fitness function to assess the computational fitness of the population. The fitness function is designed to facilitate the adaptation of individuals to the environment. In the context of macro layouts, this may include constraints such as macro overlap, macro pin orientation, and the placement of large macros on the edge.

In an actual route, it is usually connected to another point horizontally or vertically. This connection can be expressed in terms of Manhattan distances. In this article, the line length is calculated using the fitness value. The lower the fitness value, the shorter the line length, and the higher the quality of the macro layout.

D. The Basic Flow of the Algorithm in the Paper

The initial data of the chip design is initially collected, and then the macro placement constraints are combined with the fitness function in the genetic algorithm. Generating a new population follows this through continuous iteration through selection, crossover, and mutation in the genetic algorithm. The fitness value of each individual in the population is then evaluated. The optimal macro layout is identified by evaluating the line length of each layout. The layout with the shortest line length is selected as the optimal macro layout. The general algorithmic framework flow is depicted in Fig. 2.

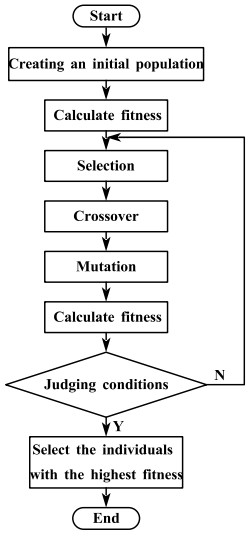

Figure 2. Framework diagram of genetic algorithm process

III. Experimental Procedure

Initially, data was collected on the chip design, including the size and location of the macros and the connections between the macros. Based on the existing chip layout data, this paper uses a genetic algorithm to simulate the initial alignment scheme. Subsequently, by adjusting the positions of the macros, the algorithm attempts to identify a macro layout with a shorter alignment length.

A. Fitness Functions and Constraint Settings

The fitness value in this paper is inversely proportional to the training iteration process, as determined by the macro layout placement constraints and the characteristics of the fitness function in the genetic algorithm. This paper employs the calculation of penalty scores to set the fitness function, with the penalty scores being added by one if a macro layout that violates the macro placement conditions is generated. The constraints in this paper are primarily as follows:

1) Macro-overlap: It is determined based on the coordinates of the macro (the default lower-left corner) and the dimensions of the macro (width and height). First, select the coordinates and dimensions of the macro from the width and height direction to retrieve the coordinates of the other macro and its dimensions. If the difference between the coordinates of the two macros in either direction of width or height is less than the larger of the two macros in terms of size, it can be set as the overlap of the two macros.

2) Pin orientation of macros: It is recommended that adjacent position macro pins be placed in a face-to-face

Authorized licensed use limited to: The University of British Columbia Library. Downloaded on April 01,2026 at 02:49:29 UTC from IEEE Xplore. Restrictions apply.

configuration, with the alignment sharing a channel. This configuration facilitates convenient data flow interaction between the macros, reduces the line length, and is illustrated in Fig. 3. In this example, the right a and b macro pins are positioned face-to-face, with shorter alignment. In the algorithm, the initial step is to select a macro through the coordinates and dimensions of the macro. This allows the neighboring macros to be determined, enabling the pin orientations of the two macros to be identified. This information is then used to select the most appropriate macro pin placement direction.

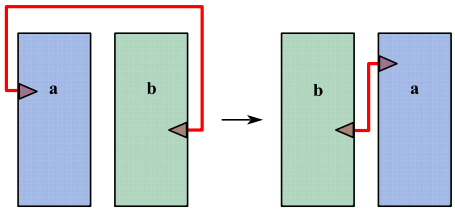

Figure 3. Schematic diagram of the effect of macro pin orientation on wire length[7]

3) Larger macro placement: In this paper, the algorithm will place macros inside the chip, but the area of large macros placed inside will have a greater impact on the alignment. According to the design of the chip area and the size of the internal macro, the design of large macro parameters prioritizes the large macro placed on the edge. If the macro area is larger than 0.003, the entire module area can be recognized as a macro, or the macro-region is in the top 3% of all macros, and its area is twice as large as the average area of all macros.

B. Fitness values and line lengths

In this paper, the length of the genetic algorithm is measured using the Manhattan distance, which is usually only available horizontally or vertically due to limitations in the manufacturing process. A Manhattan alignment can be considered a path on a grid from one point to another, where the path can only move along the grid lines.

For example, if the start points are located at coordinates $(x1, y1)$ and $(x2, y2)$, respectively, then the Manhattan alignment length L between them can be calculated using the following equation:

$$L = | x2 - x1 | + | y2 - y1 | \tag{1}$$

Here, $| x2 - x1 |$ is the horizontal distance, $| y2 - y1 |$ is the vertical distance, and the unit of line length is micro in the data in this paper.

C. Algorithmic operations

The basic steps of genetic algorithm include:

- Initialization: A random number of individuals, each representing a possible macro layout.
- Evaluation: Each individual is assessed for fitness and its line length is calculated.
- Selection: A subset of individuals is selected in some way as the population for the next generation based on the fitness value of each individual.
- Crossover: Perform crossover operations on some individuals to create new ones.
- Mutation: Mutation of some individuals to increase the diversity of the population.

Repeat the selection, crossover, and mutation until the stop condition is met.

The selection process mimics the principle of "survival of the fittest" in nature, " aiming to prioritize the reproduction of highly adapted individuals [8]. The fitness function is usually formalized as:

$$F(x) = \alpha_1 \cdot f_1(x) + \alpha_2 \cdot f_2(x) + \cdots + \alpha_n \cdot f_n(x) \tag{2}$$

$F(x)$ is the total fitness of individual x, and $f_n\ (x)$ is the ith assessment criterion. α_i is its corresponding weight, indicating the importance of different assessment criteria.

The crossover process is the primary means of creating new individuals in genetic algorithms. For example, in a single-point crossover, two chromosomes are cut at randomly selected locations, and then the segments are interchanged to create two new chromosomes. For example, suppose there are two chromosomes $A = [a_1, a_2, \dots, a_n]$ and $B = [b_1, b_2, \dots, b_n]$, then after performing a single point crossover, the new chromosome $D = [b_1, b_2, \dots, b_k, a_{k+1}, \dots, a_n]$.

The mutation process introduces new genetic variants by randomly changing one or more genes in a chromosome, thus increasing the diversity of the population and helping the algorithm move out of the local optimum and explore a wider solution space[9]. The mutation operation can be represented in a simplified way as follows: for chromosome $C = [c_1, c_2, \dots, c_n]$, a position i and a new gene value c_i' are randomly selected, then the mutated chromosome is $C'' = [c_1, c_2, \dots, c_i', \dots, c_n]$.

Finally, the genetic algorithm in this paper can continuously optimize the alignment length inside the chip during the iteration process to find the optimal macro layout that satisfies the constraints. The success of the genetic algorithm largely depends on the chromosome encoding method, the design of the fitness function, the choice of crossover and mutation strategies, and the tuning of the algorithm parameters[10].

IV. Experimental Results

In this paper, a module from integrated circuit design is used. The same settings were used for place-and-route, and finally the transition time was calculated using the STA tool. The main body of genetic algorithm in this paper is written in Python.

Fig. 4 shows the training process of the algorithm in this paper. The vertical coordinate is the fitness value, and the horizontal coordinate is the number of generations in the training population. The graph has three main lines: red represents the fitness values of the individuals with the worst adaptation in each generation. The blue line is the average of

Authorized licensed use limited to: The University of British Columbia Library. Downloaded on April 01,2026 at 02:49:29 UTC from IEEE Xplore. Restrictions apply.

the fitness values of all individuals in each generation. The green line represents the fitness value of the best individual in each generation. The curve is decreasing from generation to generation, which means that the quality of the optimal solution is gradually improving, indicating that the algorithm has found a macro layout with shorter line length and better quality.

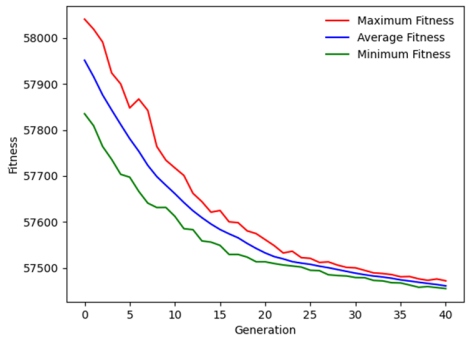

Figure 4. Genetic algorithm training process

In this paper, in the genetic algorithm, individual denotes the macro layout, fitness denotes the line length, and a decrease in fitness, i.e., line length, denotes an increasing quality of the macro layout of the algorithm. Finally, the algorithm of this paper can obtain the coordinates of each macro in the optimal macro layout and the line length obtained from the simulation.

Table I Timing comparison of different macro placements

Circuits	Pre-Opt		Post-Opt	
	trans	*num*	*trans*	*num*
cir1	-0.261	42066	-0.255	38016
cir2	-0.260	41916	-0.258	38886
cir3	-0.282	38402	-0.261	27496
cir4	-0.300	38398	-0.260	27324
cir5	-0.283	40538	-0.255	37265
mean	-0.277	40231.6	-0.258	33371.4

Table I shows the timing comparison of the same module before and after optimization using the algorithm in this paper.

In the final analysis, the length of the trace is due to the large load, which is reflected in the cell lib because the value of the total_output_net_capacitance is larger, so the transition time of the cell is larger. The length of the line is closely related to the transition time. So in the end, this paper compares different data with the transition time.

The first column of circuits in Table I are five manual macro layout modules: pre-opt is the macro layout data placed manually without the algorithm, and post-opt is the macro layout data derived using our algorithm. *Trans* is the value of transition time in nanoseconds (ns); a negative number only indicates a violation, and the smaller the absolute value, the closer the target setting *is*. *Num* is the number of transition time violation paths; the last line is the geometric mean data comparison. The table shows that the transition time violation after optimization using this paper's algorithm becomes smaller than before. The number of its violation paths is also reduced, which is, on average, 6.8% less than the transition time and 17% less than the number of paths of the manual macro layout, which also shows that this paper's algorithm is effective.

V. Conclusions

In this paper's genetic algorithm, the fitness value is the alignment length between the macros within the simulated chip, and the fitness value of the optimal individual is the layout scheme that realizes the shortest alignment length under the premise of satisfying the set constraints.

This paper identifies macro layouts with superior line lengths as the fitness value declines. Finally, by comparing the TRANSITION TIME violations, it is demonstrated that the algorithmic macro layouts proposed in this paper yield superior results to manually placed macros with better line lengths. Furthermore, the quality of the macro layouts is optimized.

Acknowledgment

This work was supported by a Project supported by ERC of ODCT and Sponsored by the CCF-Phytium Fund.

References

[1] J.-M. Lin, Y.-L. Deng, Y.-C. Yang, J.-J. Chen, and P.-C. Lu, “Dataflow-Aware Macro Placement Based on Simulated Evolution Algorithm for Mixed-Size Designs,” IEEE Trans. Very Large Scale Integr. VLSI Syst., vol. 29, no. 5, pp. 973–984, May 2021, doi: 10.1109/TVLSI.2021.3057921.

[2] J. Gu, H. Gu, K. Liu, and Z. Zhu, “An Effective Macro Placement Algorithm Based On Curiosity-Driven Reinforcement Learning,” in 2023 International Symposium of Electronics Design Automation (ISEDA), May 2023, pp. 364–368. doi: 10.1109/ISEDA59274.2023.10218370.

[3] Y.-H. Huang et al., “Routability-Driven Macro Placement with Embedded CNN-Based Prediction Model,” in 2019 Design, Automation & Test in Europe Conference & Exhibition (DATE), Mar. 2019, pp. 180–185. doi: 10.23919/DATE.2019.8715126.

[4] Y. Chen, Z. Wen, Y. Liang, and Y. Lin, "Stronger Mixed-Size Placement Backbone Considering Second-Order Information," in 2023 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), Oct. 2023, pp. 1–9. doi: 10.1109/ICCAD57390.2023.10323700.

[5] Y.-F. Chen, C.-C. Huang, C.-H. Chiou, Y.-W. Chang, and C.-J. Wang, “Routability-driven blockage-aware macro placement,” in 2014 51st ACM/EDAC/IEEE Design Automation Conference (DAC), Jun. 2014, pp. 1–6. doi: 10.1145/2593069.2593206.

[6] J. Lu et al., “ePlace-MS: Electrostatics-Based Placement for Mixed-Size Circuits,” IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst., vol. 34, no. 5, pp. 685–698, May 2015, doi: 10.1109/TCAD.2015.2391263.

[7] C.-C. Tan, W.-H. Chen, Z.-M. Yeh, and W.-K. Cheng, “Routability-driven macro placement considering pins location,” in 2017 International Conference on Electron Devices and Solid-State Circuits (EDSSC), Oct. 2017, pp. 1–2. doi: 10.1109/EDSSC.2017.8126485.

[8] A. Anwaar, A. Ashraf, W. H. K. Bangyal, and M. Iqbal, “Genetic Algorithms: Brief review on Genetic Algorithms for Global Optimization Problems,” in 2022 Human-Centered Cognitive Systems (HCCS), Dec. 2022, pp. 1–6. doi: 10.1109/HCCS55241.2022.10090327.

[9] Y. Zhao and B. Li, “Design of hybrid genetic algorithm based on ant colony algorithm,” in 2022 4th International Conference on Artificial Intelligence and Advanced Manufacturing (AIAM), Oct. 2022, pp. 834–837. doi: 10.1109/AIAM57466.2022.00169.

[10] R. V. Solopov and A. S. Samulchenkov, “Development of Fitness Function of Electrical Network Evolutionary Model for Computation of Settling Behavior by Method of Genetic Algorithm,” in 2019 International Conference on Industrial Engineering, Applications and Manufacturing (ICIEAM), Mar. 2019, pp. 1–5. doi: 10.1109/ICIEAM.2019.8742968.

Authorized licensed use limited to: The University of British Columbia Library. Downloaded on April 01,2026 at 02:49:29 UTC from IEEE Xplore. Restrictions apply.